



Decision Tree

Dr. Prasenjit Dey,
Assistant Professor,
NIT Rourkela,
Odisha, India

Decision Trees

- Each path from root to a leaf defines a region R_m of the input space
- Let $\{(x_i, t_i)\}$ be the training examples belonging to R_m

R_m =region defined by a sequence of splits

Classification tree:

- Discrete output
- Leaf value y^m typically set to the most common value (mode) in $\{t_i\}$

Regression tree:

- Continuous output
- Leaf value y^m typically set to the mean of $\{t_i\}$

Examples of Classification Task

Task	Attribute set, x	Class label, y
Categorizing email messages	Features extracted from email message header and content	spam or non-spam
Identifying tumor cells	Features extracted from x-rays or MRI scans	malignant or benign cells
Cataloging galaxies	Features extracted from telescope images	Elliptical, spiral, or irregular-shaped galaxies

General Approach for Building Classification Model

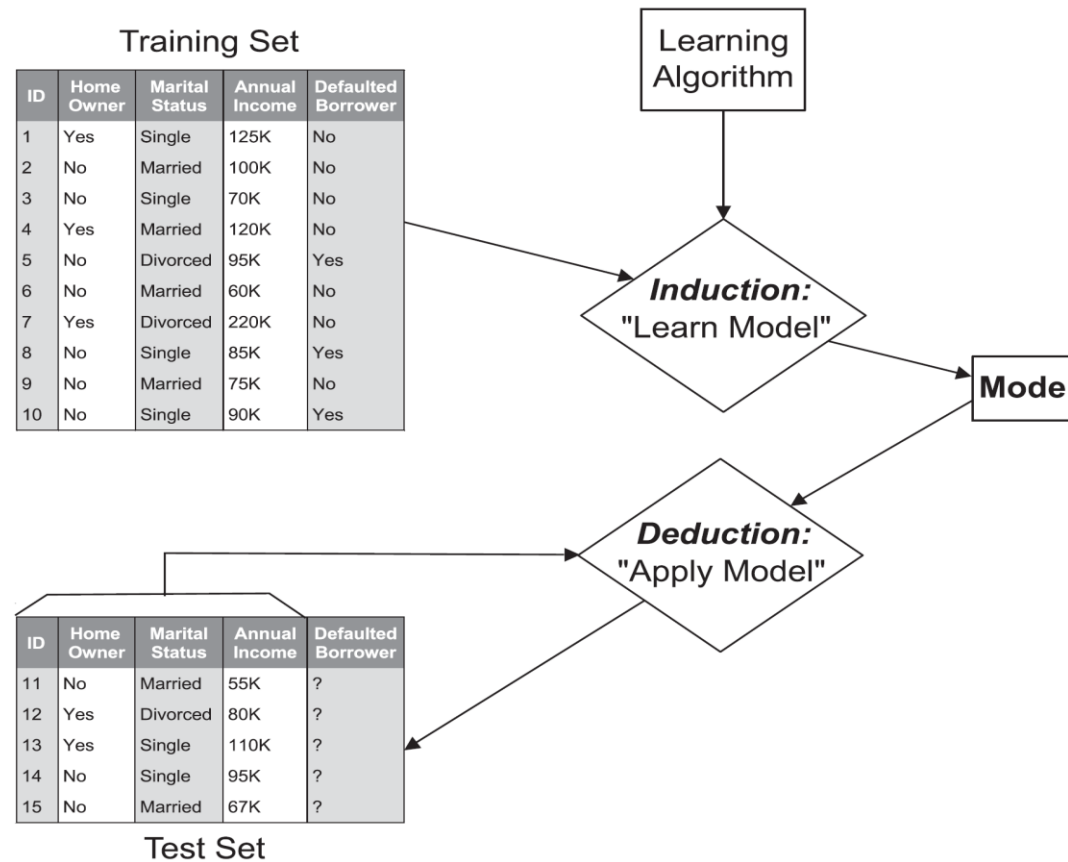
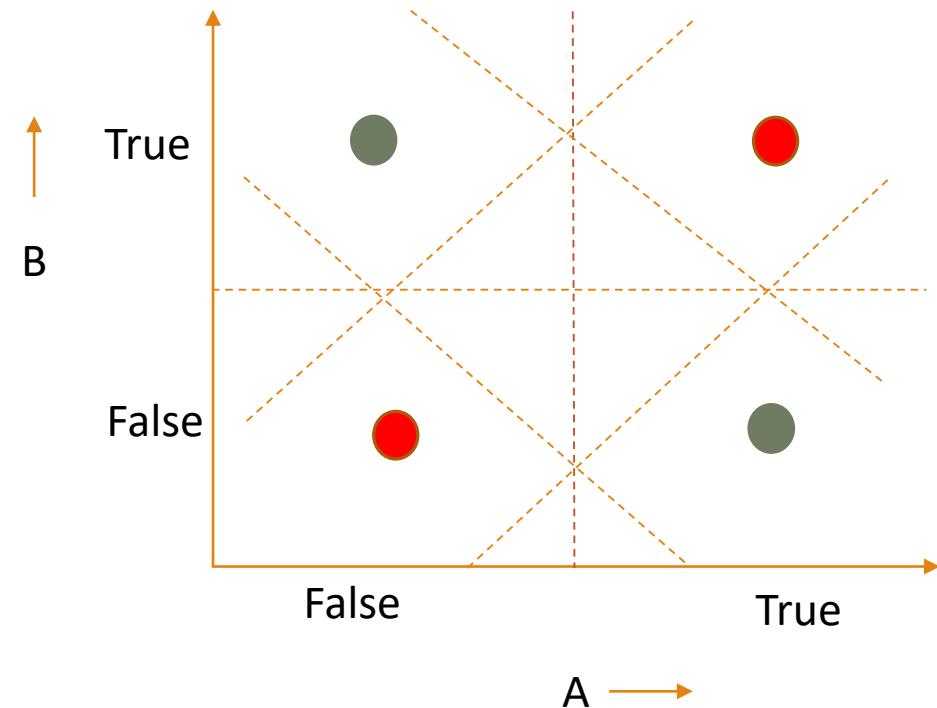
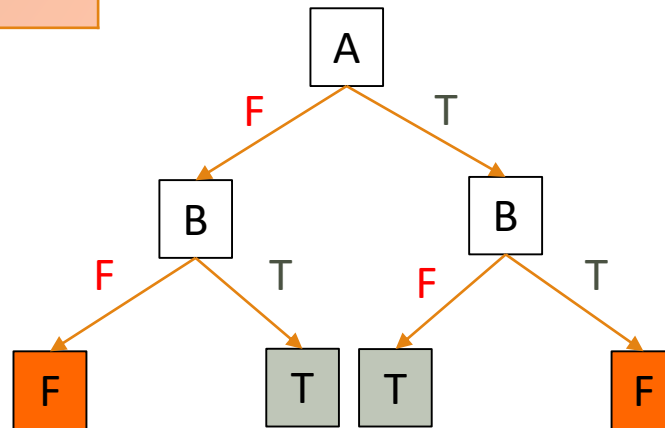


Figure 3.3. General framework for building a classification model.

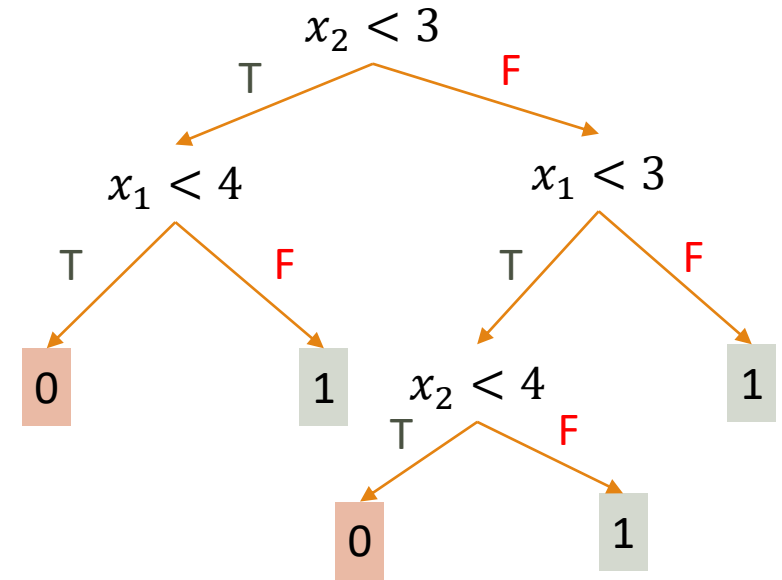
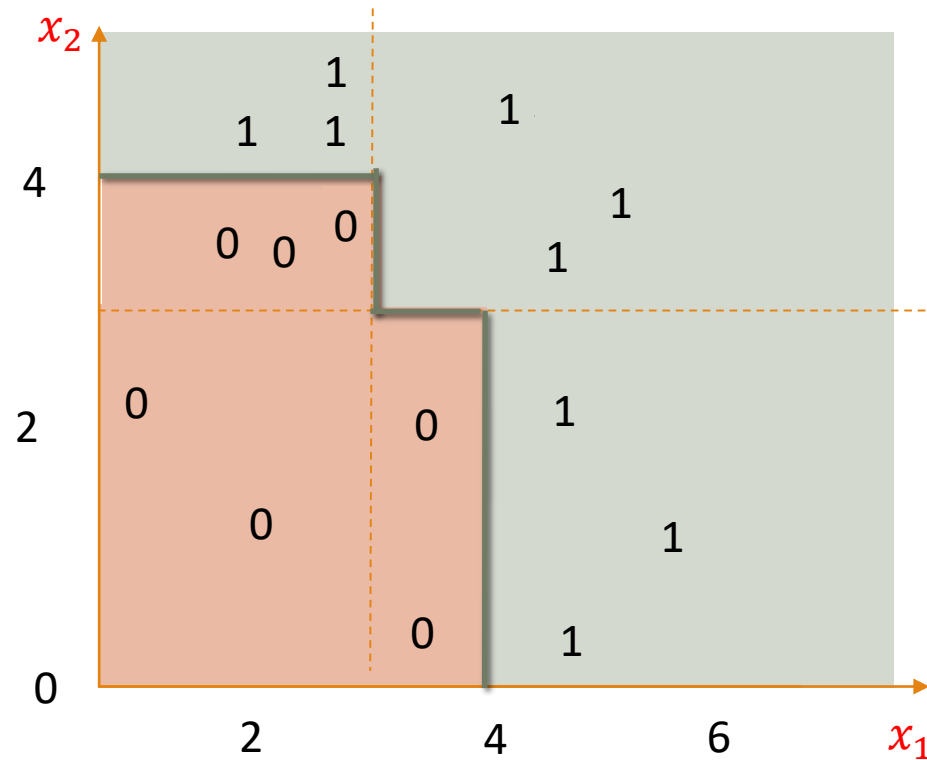
Decision Tree: Expressiveness

Example: XOR

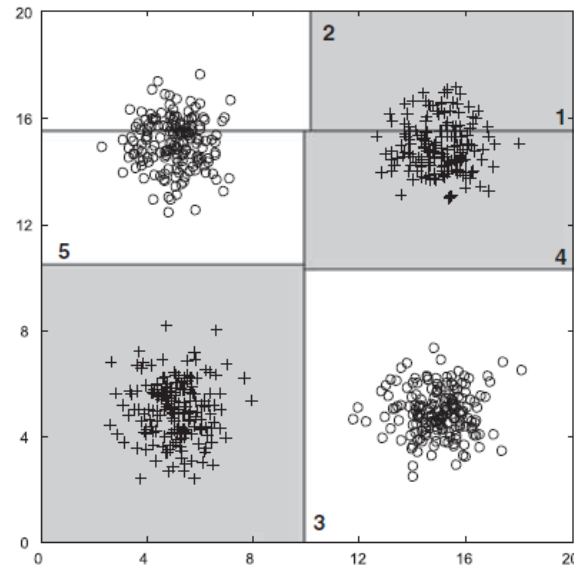
A	B	A XOR B
F	F	F
F	T	T
T	F	T
T	T	F



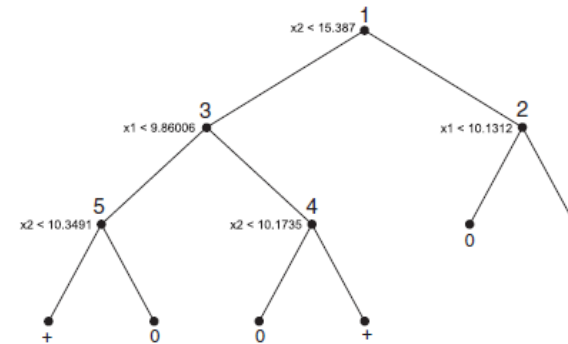
Decision trees divide the feature space into hyper rectangles.
Each region is labelled with one label



Handling interactions



(a) Decision boundary for tree with 6 leaf nodes.



(b) Decision tree with 6 leaf nodes.

Figure 3.28. Decision tree with 6 leaf nodes using X and Y as attributes. Splits have been numbered from 1 to 5 in order of other occurrence in the tree.

Decision Tree Learning Algorithm

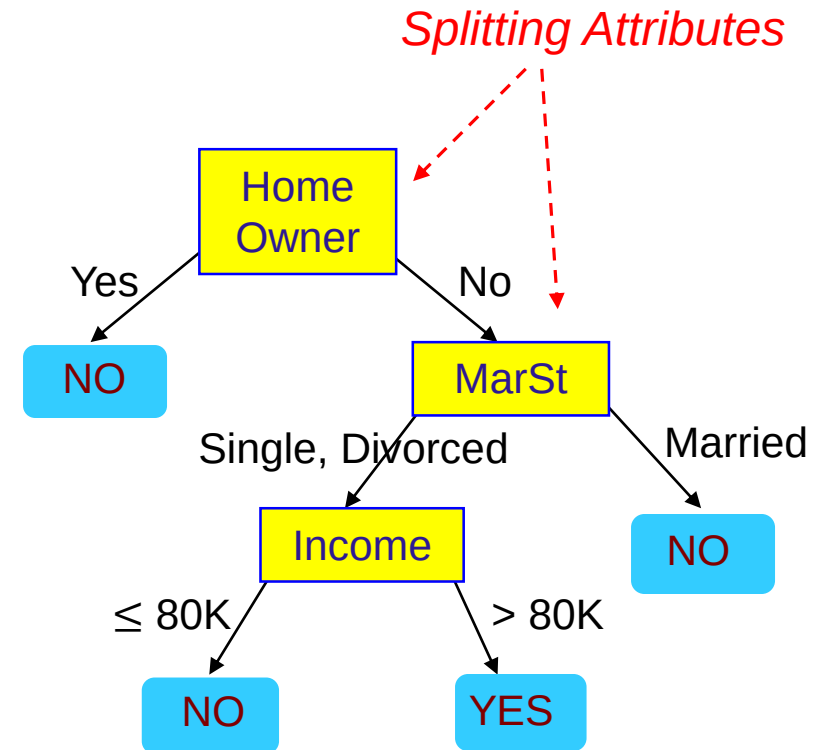
- Use a **greedy heuristic** to construct the tree
 - Start with root node
 - At each step, split on the best attribute
 - Repeat recursively for each subset
- How do we choose the best attribute?
 - Use **entropy**
 - Select attribute with highest Information Gain (or lowest Gini index)

Example of a Decision Tree

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

categorical
categorical
continuous
class

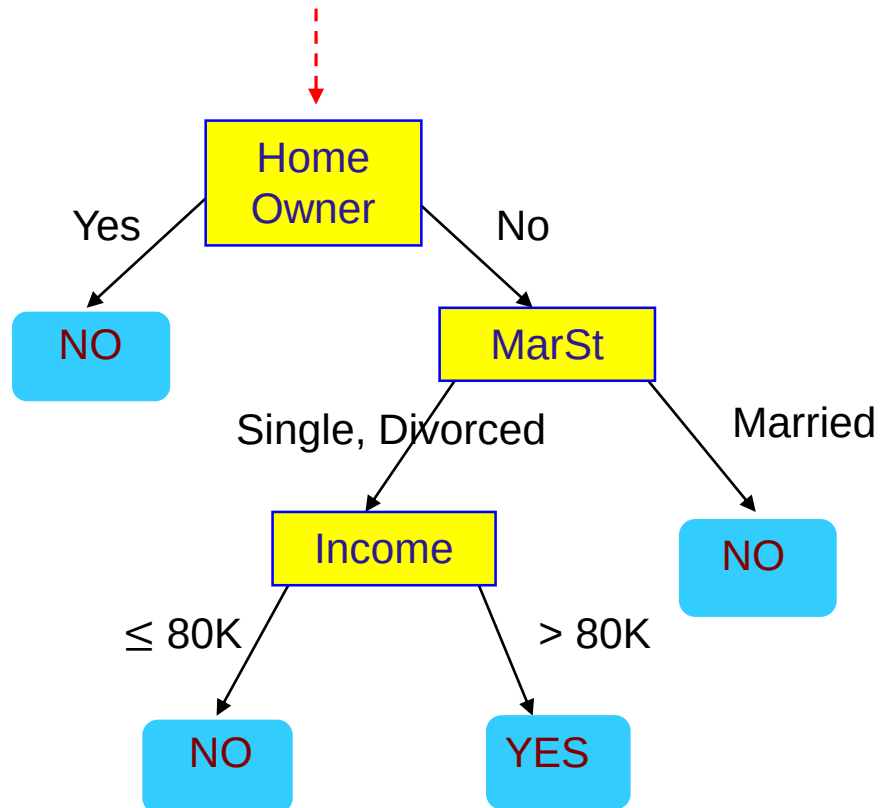
Training Data



Model: Decision Tree

Apply Model to Test Data

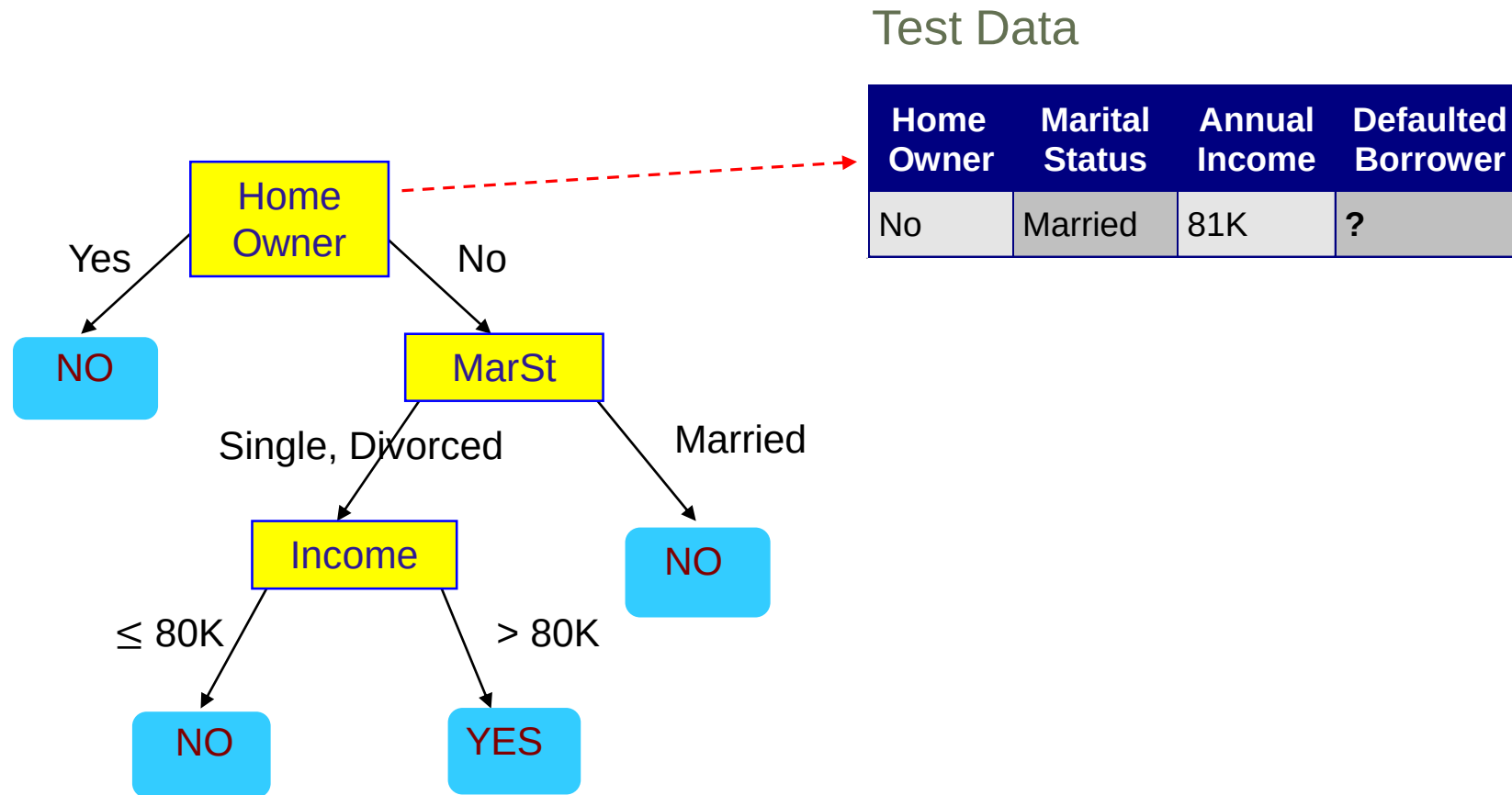
Start from the root of tree.



Test Data

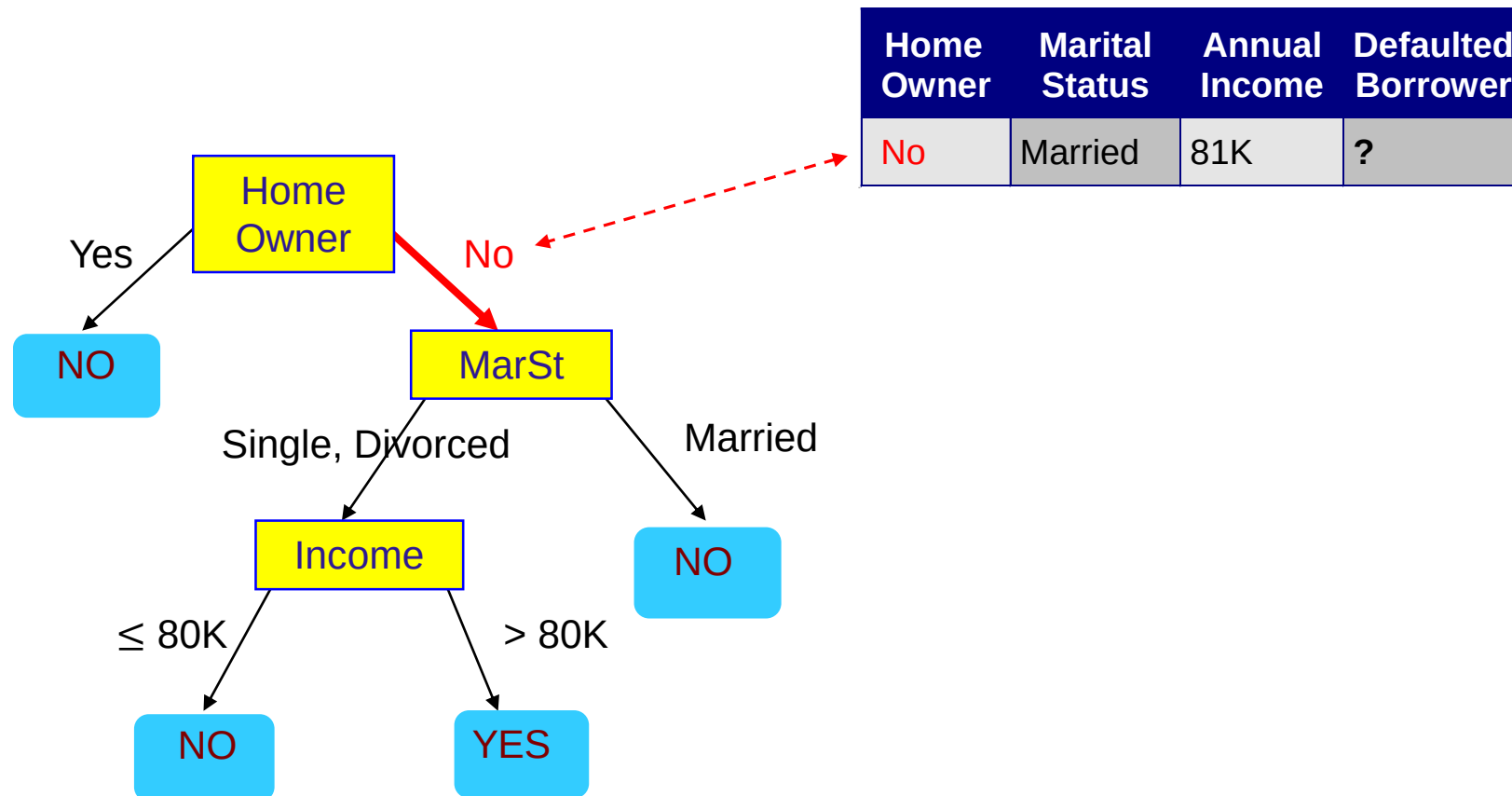
Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	81K	?

Apply Model to Test Data



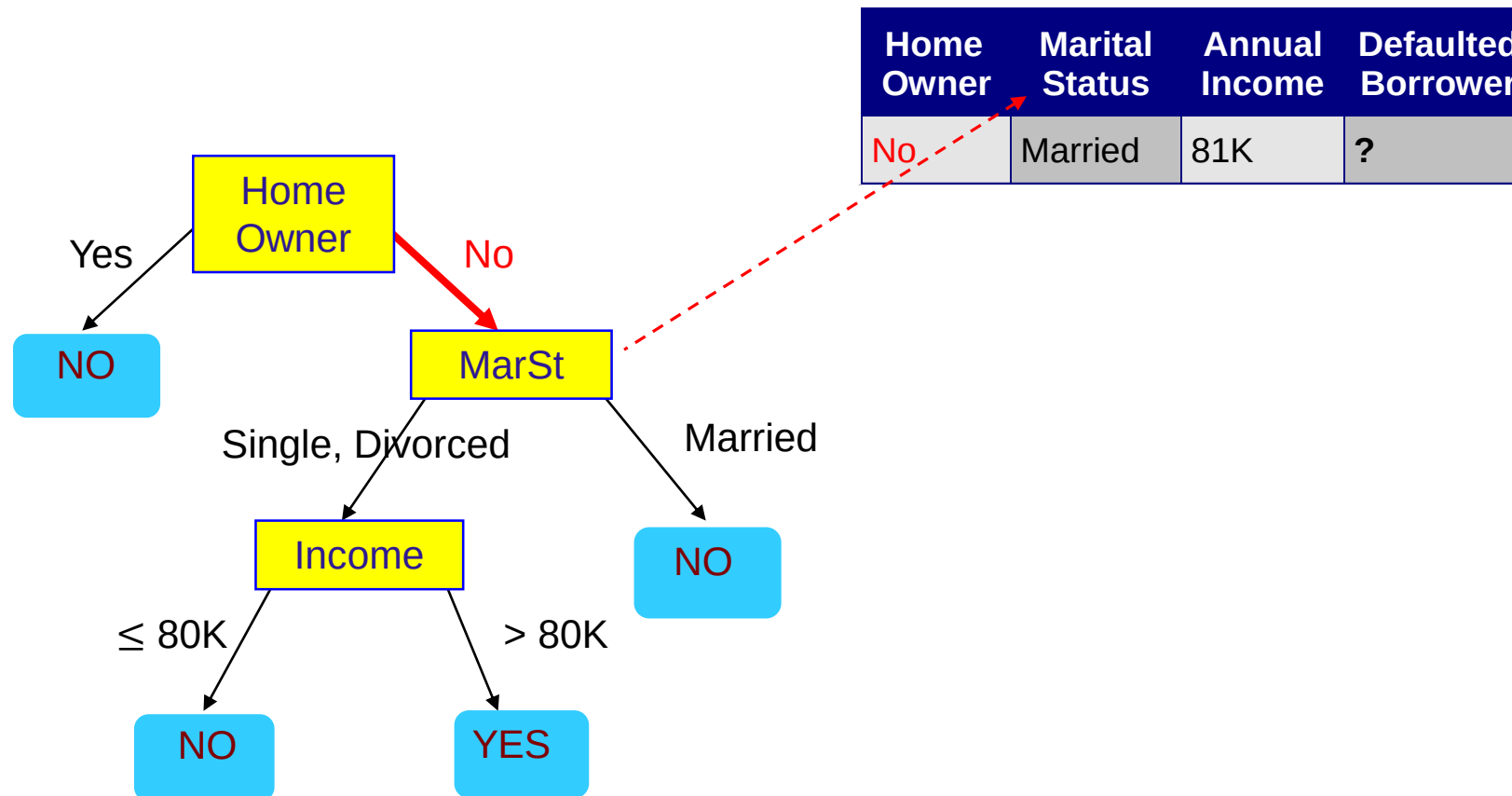
Apply Model to Test Data

Test Data



Apply Model to Test Data

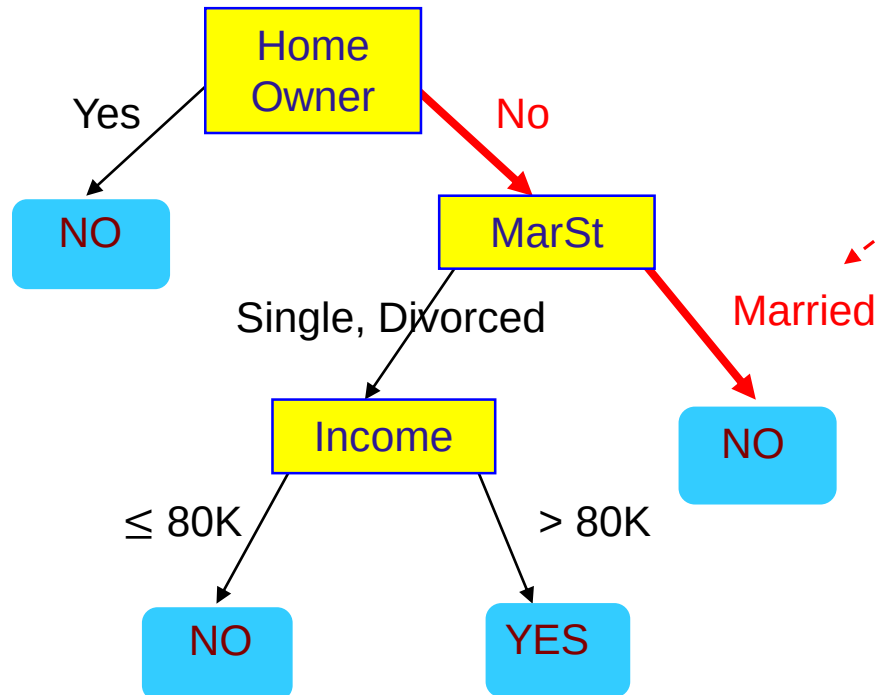
Test Data



Apply Model to Test Data

Test Data

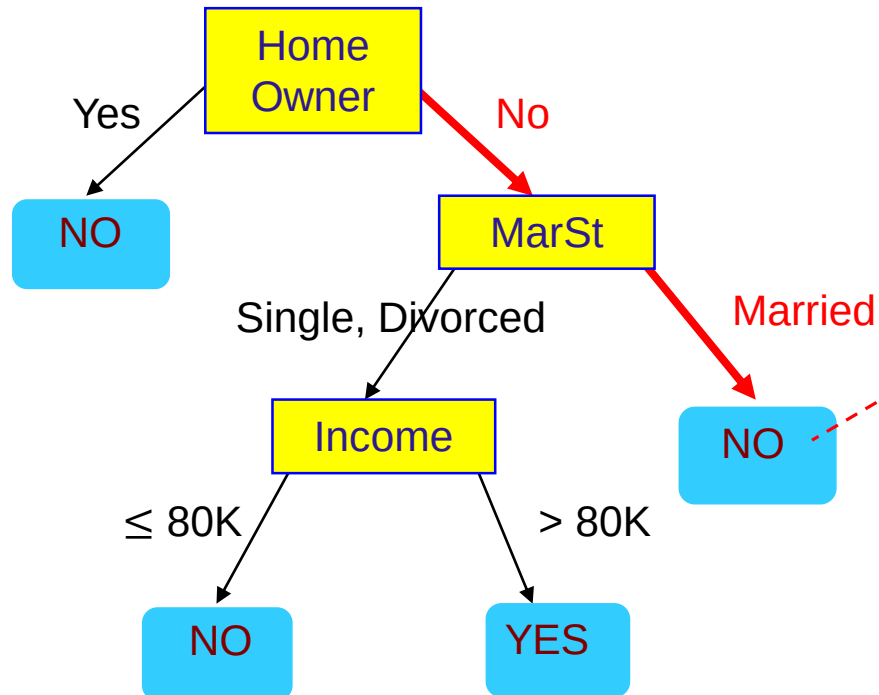
Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	81K	?



Apply Model to Test Data

Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	81K	?

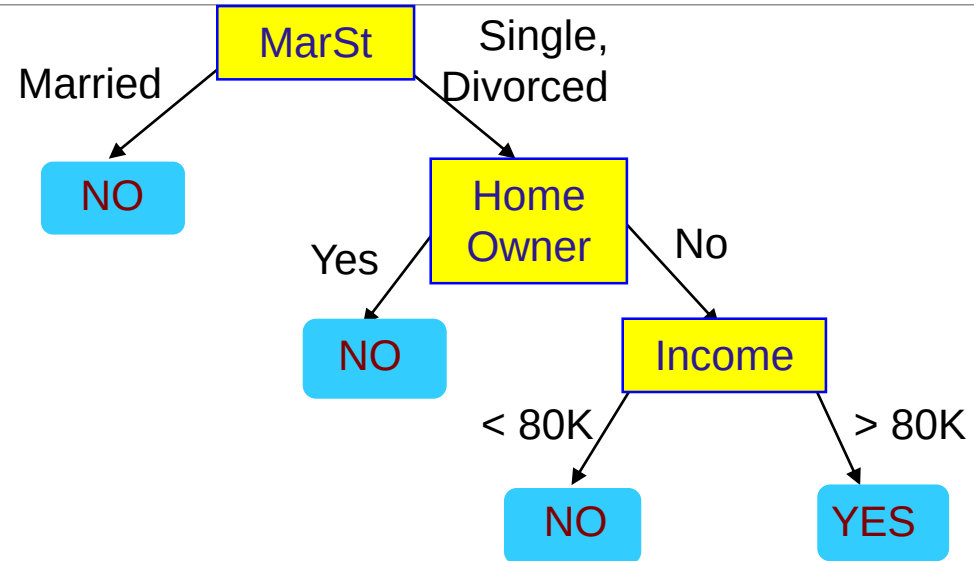


Assign Defaulted to
"No"

Another Example of Decision Tree

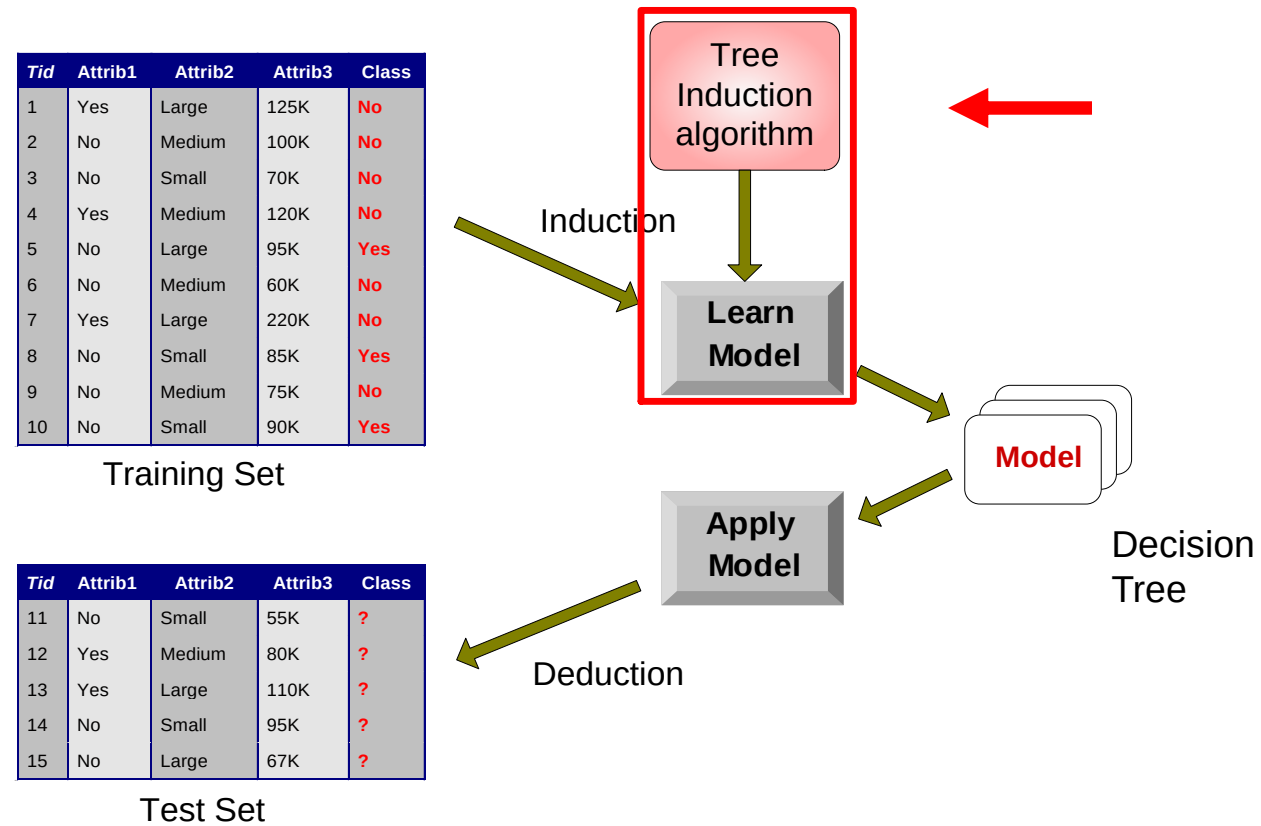
categorical
categorical
continuous
class

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes



There could be more than one tree that fits the same data!

Decision Tree Classification Task



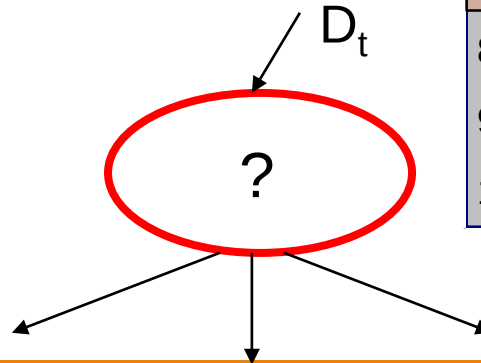
Decision Tree Induction

Many Algorithms:

- Hunt's Algorithm (one of the earliest)
- CART
- ID3, C4.5
- SLIQ, SPRINT

General Structure of Hunt's Algorithm

- Let D_t be the set of training records that reach a node t
- General Procedure:
 - If D_t contains records that belong the same class y_t , then t is a leaf node labeled as y_t
 - If D_t contains records that belong to more than one class, use an attribute test to split the data into smaller subsets. Recursively apply the procedure to each subset.



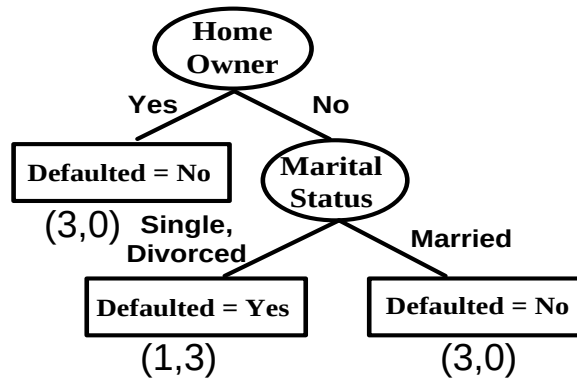
ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Hunt's Algorithm

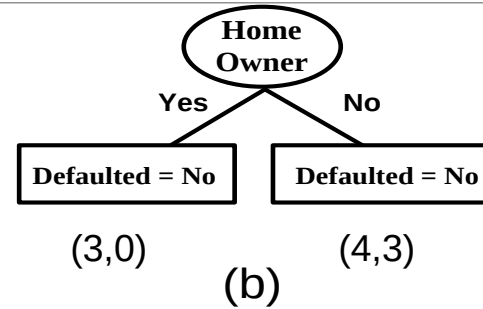


(7,3)

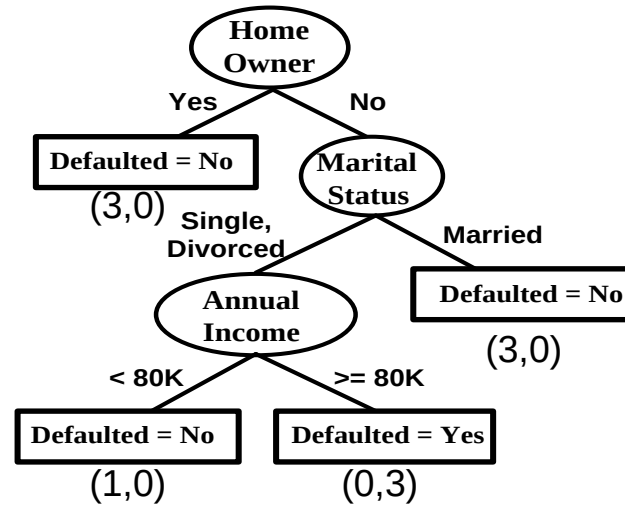
(a)



(c)



(b)



(d)

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Design Issues of Decision Tree Induction

- How should training records be split?
 - Method for expressing test condition
 - ◆ depending on attribute types
 - Measure for evaluating the goodness of a test condition

- How should the splitting procedure stop?
 - Stop splitting if all the records belong to the same class or have identical attribute values
 - Early termination

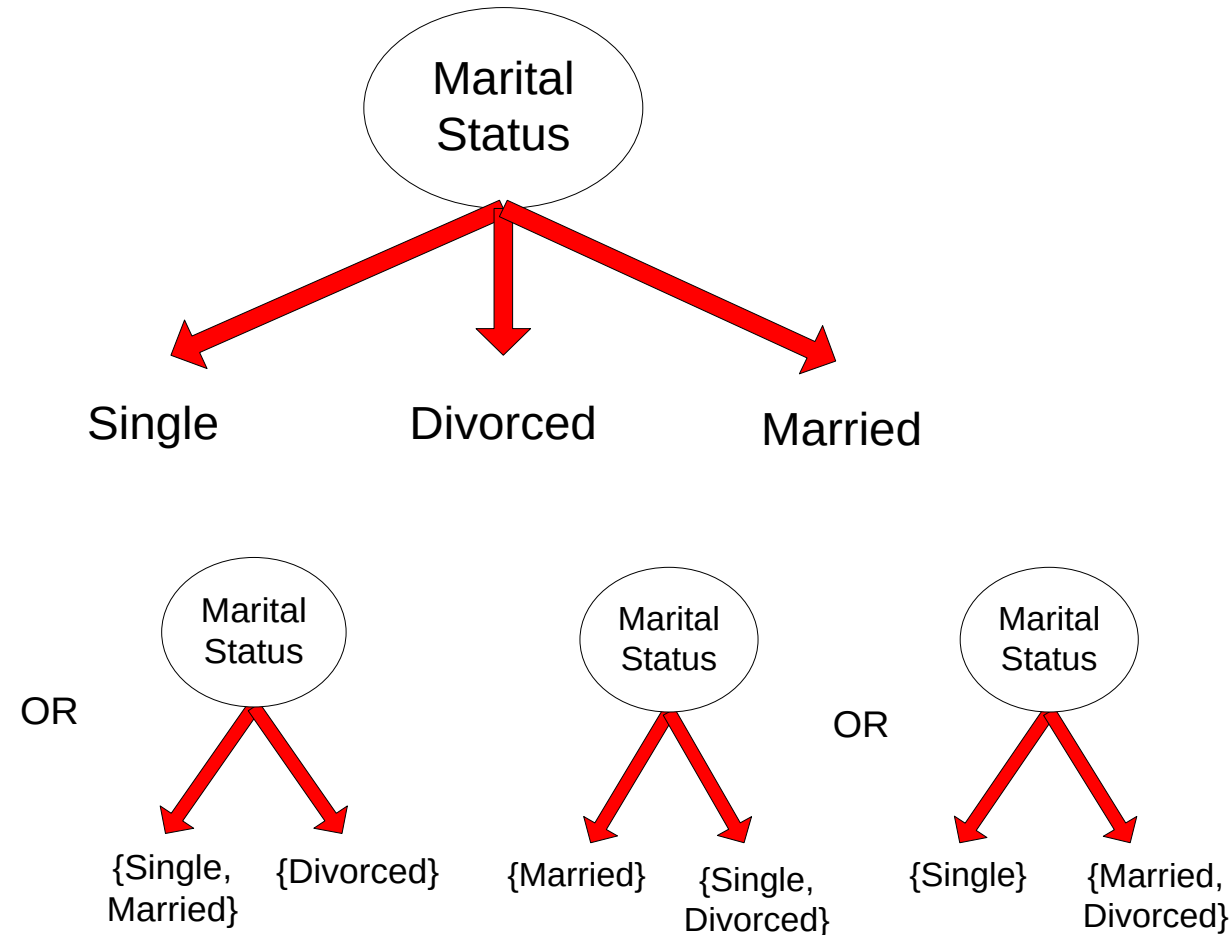
Methods for Expressing Test Conditions

- Depends on attribute types

- Binary
- Nominal
- Ordinal
- Continuous

Test Condition for Nominal Attributes

- **Multi-way split:**
 - Use as many partitions as distinct values.
- **Binary split:**
 - Divides values into two subsets



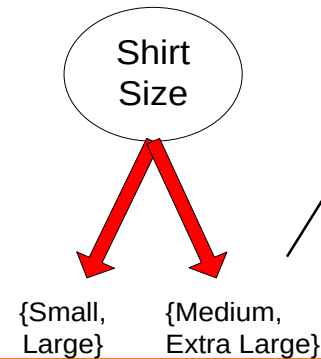
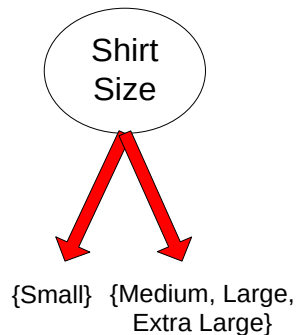
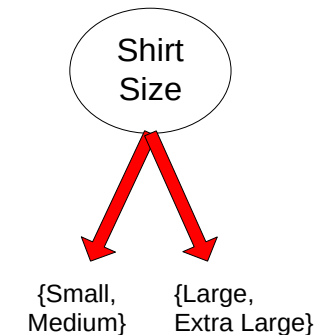
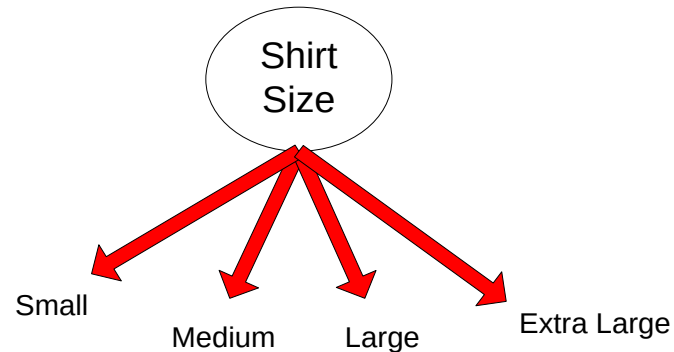
Test Condition for Ordinal Attributes

Multi-way split:

- Use as many partitions as distinct values

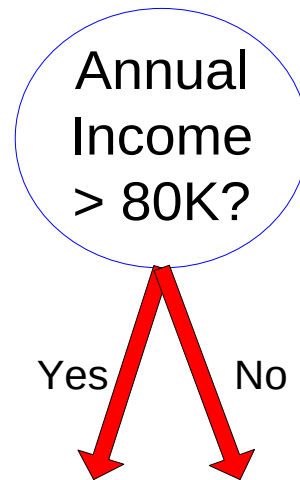
Binary split:

- Divides values into two subsets
- Preserve order property among attribute values

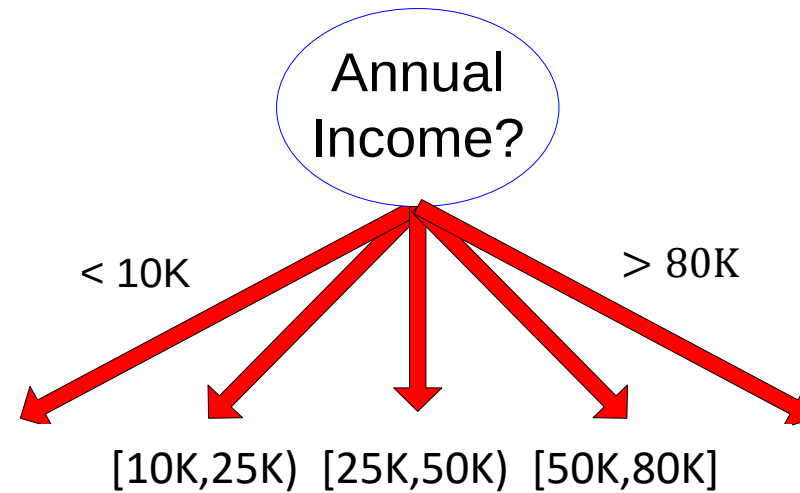


This grouping violates order property

Test Condition for Continuous Attributes



(i) Binary split



(ii) Multi-way split

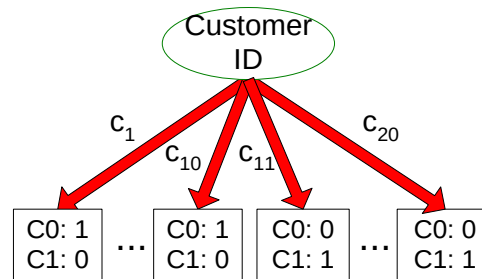
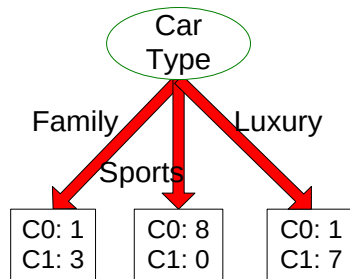
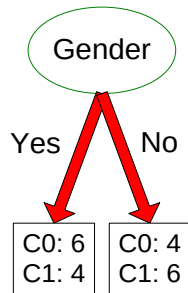
Splitting Based on Continuous Attributes

Different ways of handling

- **Discretization** to form an ordinal categorical attribute
 - Ranges can be found by equal interval bucketing, equal frequency bucketing (percentiles), or clustering.
 - Static – discretize once at the beginning
 - Dynamic – repeat at each node
- **Binary Decision**: $(A < v)$ or $(A \geq v)$
 - consider all possible splits and finds the best cut
 - can be more compute intensive

How to determine the Best Split

Before Splitting: 10 records of class 0,
10 records of class 1



Customer Id	Gender	Car Type	Shirt Size	Class
1	M	Family	Small	C0
2	M	Sports	Medium	C0
3	M	Sports	Medium	C0
4	M	Sports	Large	C0
5	M	Sports	Extra Large	C0
6	M	Sports	Extra Large	C0
7	F	Sports	Small	C0
8	F	Sports	Small	C0
9	F	Sports	Medium	C0
10	F	Luxury	Large	C0
11	M	Family	Large	C1
12	M	Family	Extra Large	C1
13	M	Family	Medium	C1
14	M	Luxury	Extra Large	C1
15	F	Luxury	Small	C1
16	F	Luxury	Small	C1
17	F	Luxury	Medium	C1
18	F	Luxury	Medium	C1
19	F	Luxury	Medium	C1
20	F	Luxury	Large	C1

Which test condition is the best?

How to determine the Best Split

| Greedy approach:

- Nodes with **pur**er class distribution are preferred

| Need a measure of node impurity:

C0: 5
C1: 5

High degree of impurity

C0: 9
C1: 1

Low degree of impurity

Measures of Node Impurity

| Gini Index

$$Gini\ Index = 1 - \sum_{i=0}^{c-1} p_i(t)^2$$

Where $p_i(t)$ is the probability of class i at node t , and c is the total number of classes

| Entropy

$$Entropy = - \sum_{i=0}^{c-1} p_i(t) \log_2 p_i(t)$$

| Misclassification error

$$Classification\ error = 1 - \max[p_i(t)]$$

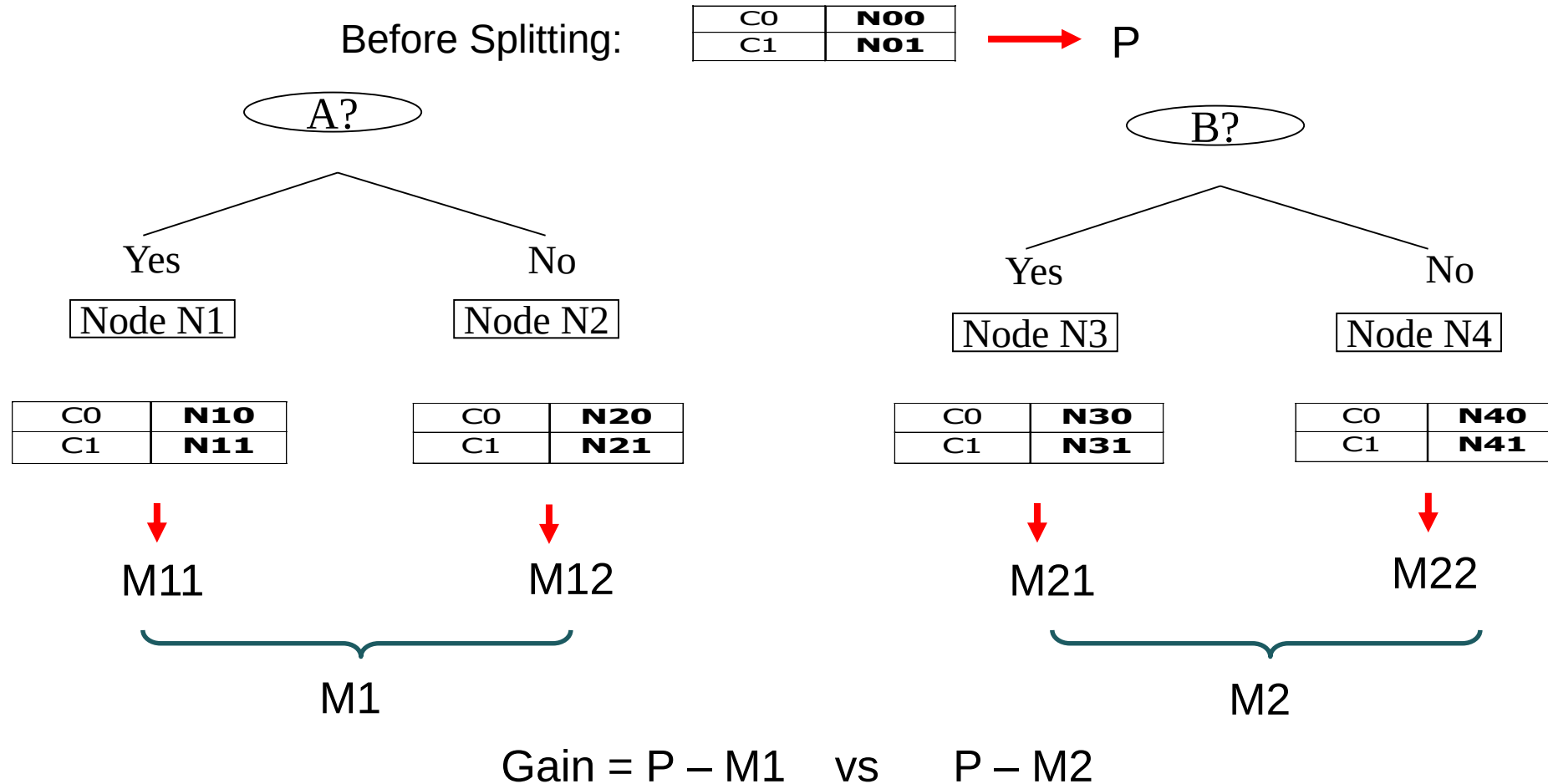
Finding the Best Split

1. Compute impurity measure (P) before splitting
2. Compute impurity measure (M) after splitting
 - | Compute impurity measure of each child node
 - | M is the weighted impurity of child nodes
3. Choose the attribute test condition that produces the highest gain

$$\text{Gain} = P - M$$

or equivalently, lowest impurity measure after splitting (M)

Finding the Best Split



1. Measure of Impurity: GINI

Gini Index for a given node t

$$Gini\ Index = 1 - \sum_{i=0}^{c-1} p_i(t)^2$$

Where $p_i(t)$ is the probability of class i at node t , and c is the total number of classes

- Maximum of $1 - 1/c$ when records are equally distributed among all classes, implying the maximum impurity (worst split)
- Minimum of 0 when all records belong to one class, implying the pure node (best split) for classification
- Gini index is used in decision tree algorithms such as CART, SLIQ, SPRINT

Measure of Impurity: GINI

Gini Index for a given node t :

$$Gini\ Index = 1 - \sum_{i=0}^{c-1} p_i(t)^2$$

- For 2-class problem (p, 1 – p):
 - $GINI = 1 - p^2 - (1 - p)^2 = 2p(1-p)$

C1	0
C2	6
Gini=0.000	

C1	1
C2	5
Gini=0.278	

C1	2
C2	4
Gini=0.444	

C1	3
C2	3
Gini=0.500	

Computing Gini Index of a Single Node

$$Gini\ Index = 1 - \sum_{i=0}^{c-1} p_i(t)^2$$

C1	0
C2	6

$$P(C1) = 0/6 = 0 \quad P(C2) = 6/6 = 1$$

$$Gini = 1 - P(C1)^2 - P(C2)^2 = 1 - 0 - 1 = 0$$

C1	1
C2	5

$$P(C1) = 1/6 \quad P(C2) = 5/6$$

$$Gini = 1 - (1/6)^2 - (5/6)^2 = 0.278$$

C1	2
C2	4

$$P(C1) = 2/6 \quad P(C2) = 4/6$$

$$Gini = 1 - (2/6)^2 - (4/6)^2 = 0.444$$

Computing Gini Index for a Collection of Nodes

- When a node p is split into k partitions (children)

$$Gini_{split} = \sum_{i=1}^k \frac{n_i}{n} Gini(t_i)$$

where, n_i = number of records at child i

n = number of records at parent node p

Select split with lowest $Gini_{split}$

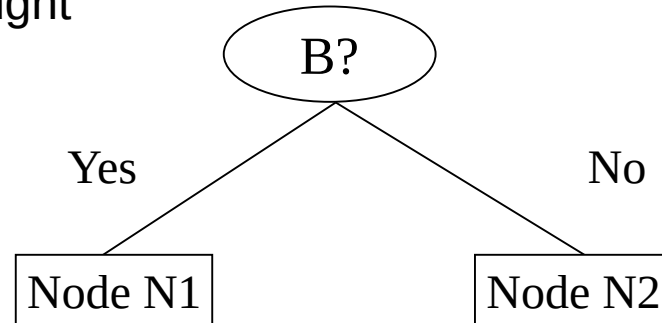
Binary Attributes: Computing GINI Index

- Splits into two partitions (child nodes)
- Effect of Weighing partitions:
 - Larger and purer partitions are sought

$$\begin{aligned} \text{Gini}(N1) &= 1 - (5/6)^2 - (1/6)^2 \\ &= 0.278 \end{aligned}$$

$$\begin{aligned} \text{Gini}(N2) &= 1 - (2/6)^2 - (4/6)^2 \\ &= 0.444 \end{aligned}$$

	N1	N2
C1	5	2
C2	1	4
Gini=0.361		



$$\begin{aligned} \text{Gini}_{split} &= (6/12) * 0.278 + (6/12) * 0.444 \\ &= 0.361 \end{aligned}$$

	Parent
C1	7
C2	5
Gini = 0.486	

$$\begin{aligned} \text{Gini}(\text{Parent}) &= 1 - [P(C1)^2 + P(C2)^2] \\ &= 1 - \left[\left(\frac{7}{12} \right)^2 + \left(\frac{5}{12} \right)^2 \right] \\ &= 0.468 \end{aligned}$$

$$\begin{aligned} \text{Gain} &= \text{Gini}(\text{Parent}) - \text{Gini}_{split} \\ &= 0.486 - 0.361 = 0.125 \end{aligned}$$

Categorical Attributes: Computing Gini Index

- | For each distinct value, gather counts for each class in the dataset
- | Use the count matrix to make decisions

Multi-way split

	CarType		
	Family	Sports	Luxury
C1	1	8	1
C2	3	0	7
Gini	0.163		

Two-way split
(find best partition of values)

	CarType	
	{Sports, Luxury}	{Family}
C1	9	1
C2	7	3
Gini	0.468	

	CarType	
	{Sports}	{Family, Luxury}
C1	8	2
C2	0	10
Gini	0.167	

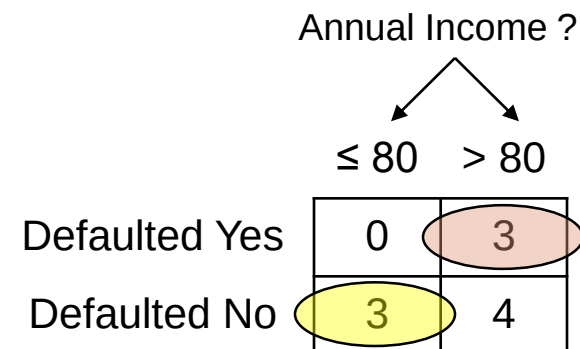
Which of these is the best?

Lower Gini $\Rightarrow$ better split
Multi-way split (Gini = 0.163) is best

Continuous Attributes: Computing Gini Index

- | Use Binary Decisions based on one value
- | Several Choices for the splitting value
 - Number of possible splitting values
= Number of distinct values
- | Each splitting value has a count matrix associated with it
 - Class counts in each of the partitions, $A \leq v$ and $A > v$
- | Simple method to choose best v
 - For each v , scan the database to gather count matrix and compute its Gini index
 - Computationally Inefficient! Repetition of work.

ID	Home Owner	Marital Status	Annual Income	Defaulted
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes



Continuous Attributes: Computing Gini Index...

- For efficient computation: for each attribute,
 - Sort the attribute on values
 - Linearly scan these values, each time updating the count matrix and computing gini index
 - Choose the split position that has the least gini index

Cheat		No		No		No		Yes		Yes		Yes		No		No		No		No			
Sorted Values →		Annual Income																					
		60		70		75		85		90		95		100		120		125		220			
		55		65		72		80		87		92		97		110		122		172		230	
		<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>
Yes		0	3	0	3	0	3	0	3	1	2	2	1	3	0	3	0	3	0	3	0	3	0
No		0	7	1	6	2	5	3	4	3	4	3	4	3	4	4	3	5	2	6	1	7	0
Gini		0.420		0.400		0.375		0.343		0.417		0.400		<u>0.300</u>		0.343		0.375		0.400		0.420	

Continuous Attributes: Computing Gini Index...

- For efficient computation: for each attribute,
 - Sort the attribute on values
 - Linearly scan these values, each time updating the count matrix and computing gini index
 - Choose the split position that has the least gini index

Sorted Values Split Positions	Cheat	No		No		No		Yes		Yes		Yes		No		No		No		No		
	Annual Income																					
	60		70		75		85		90		95		100		120		125		220			
	55		65		72		80		87		92		97		110		122		172		230	
	<= >		<= >		<= >		<= >		<= >		<= >		<= >		<= >		<= >		<= >		<= >	
	Yes	0	3	0	3	0	3	0	3	1	2	2	1	3	0	3	0	3	0	3	0	3
No	0	7	1	6	2	5	3	4	3	4	3	4	3	4	4	3	5	2	6	1	7	0
Gini	0.420		0.400		0.375		0.343		0.417		0.400		<u>0.300</u>		0.343		0.375		0.400		0.420	

Continuous Attributes: Computing Gini Index...

- For efficient computation: for each attribute,
 - Sort the attribute on values
 - Linearly scan these values, each time updating the count matrix and computing gini index
 - Choose the split position that has the least gini index

Choose the split position that has the least gini index

Cheat	No	No	No	Yes	Yes	Yes	No	No	No	No										
	Annual Income																			
Sorted Values	60	70	75	85	90	95	100	120	125	220										
Split Positions	55	65	72	80	87	92	97	110	122	172	230									
	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>								
Yes	0	3	0	3	0	3	1	2	2	1	3	0	3	0	3	0				
No	0	7	1	6	2	5	3	4	3	4	3	4	4	3	5	2	6	1	7	0
Gini	0.420	0.400	0.375	0.343	0.417	0.400	0.300	0.343	0.375	0.400	0.420									

Continuous Attributes: Computing Gini Index...

- For efficient computation: for each attribute,
 - Sort the attribute on values
 - Linearly scan these values, each time updating the count matrix and computing gini index
 - Choose the split position that has the least gini index

Cheat		No		No		No		Yes		Yes		Yes		No		No		No		No			
Sorted Values Split Positions		Annual Income																					
		60		70		75		85		90		95		100		120		125		220			
		55		65		72		80		87		92		97		110		122		172		230	
		<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>
Yes		0	3	0	3	0	3	0	3	1	2	2	1	3	0	3	0	3	0	3	0	3	0
No		0	7	1	6	2	5	3	4	3	4	3	4	3	4	4	3	5	2	6	1	7	0
Gini		0.420		0.400		0.375		0.343		0.417		0.400		0.300		0.343		0.375		0.400		0.420	

Continuous Attributes: Computing Gini Index...

- For efficient computation: for each attribute,
 - Sort the attribute on values
 - Linearly scan these values, each time updating the count matrix and computing gini index
 - Choose the split position that has the least gini index

Cheat		No		No		No		Yes		Yes		Yes		No		No		No		No			
Sorted Values Split Positions	→	Annual Income																					
		60		70		75		85		90		95		100		120		125		220			
		55		65		72		80		87		92		97		110		122		172		230	
		<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>
	Yes	0	3	0	3	0	3	0	3	1	2	2	1	3	0	3	0	3	0	3	0	3	0
	No	0	7	1	6	2	5	3	4	3	4	3	4	3	4	4	3	5	2	6	1	7	0
	Gini	0.420		0.400		0.375		0.343		0.417		0.400		<u>0.300</u>		0.343		0.375		0.400		0.420	

2. Measure of Impurity: Entropy

Entropy at a given node t

$$Entropy = - \sum_{i=0}^{c-1} p_i(t) \log_2 p_i(t)$$

Where $p_i(t)$ is the probability of class i at node t , and c is the total number of classes

- ◆ Maximum of $\log_2 c$ when records are equally distributed among all classes, implying the maximum uncertainty (worst case).
- ◆ Minimum of 0 when all records belong to one class, implying pure node (no uncertainty) for classification.
- ◆ Higher entropy $\rightarrow$ more impurity

— Entropy based computations are quite similar to the GINI index computations

Computing Entropy of a Single Node

$$Entropy = - \sum_{i=0}^{c-1} p_i(t) \log_2 p_i(t)$$

C1	0
C2	6

$$P(C1) = 0/6 = 0 \quad P(C2) = 6/6 = 1$$

$$Entropy = - 0 \log 0 - 1 \log 1 = - 0 - 0 = 0(\text{pure node})$$

C1	1
C2	5

$$P(C1) = 1/6 \quad P(C2) = 5/6$$

$$Entropy = - (1/6) \log_2 (1/6) - (5/6) \log_2 (5/6) = 0.65$$

C1	2
C2	4

$$P(C1) = 2/6 \quad P(C2) = 4/6$$

$$Entropy = - (2/6) \log_2 (2/6) - (4/6) \log_2 (4/6) = 0.92$$

Computing Information Gain After Splitting

| Information Gain:

$$\text{Information Gain} = \text{Entropy}(p) - \sum_{i=1}^k \frac{n_i}{n} \text{Entropy}(i)$$

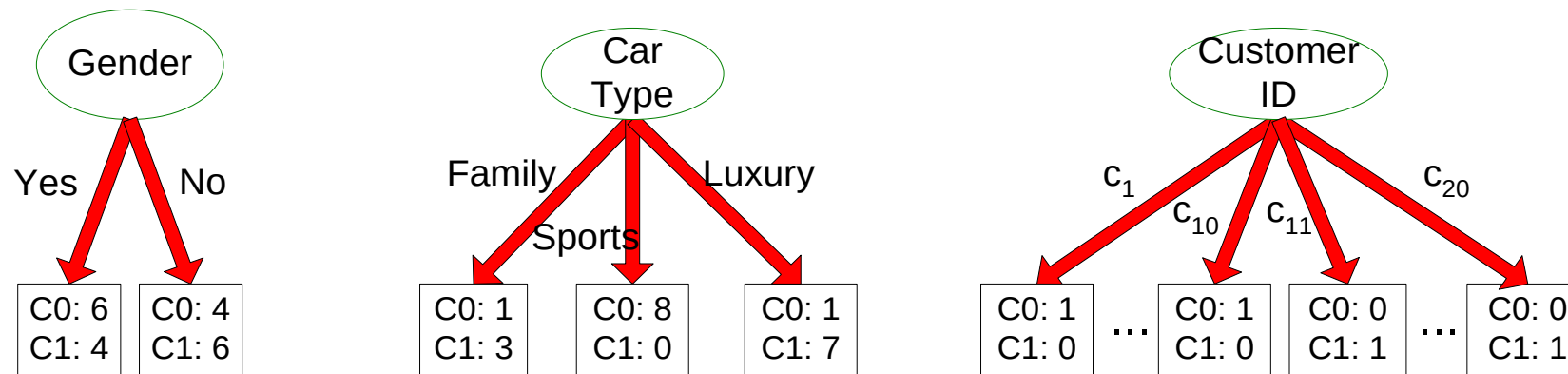
Parent Node, p is split into k partitions (children)

n_i is number of records in child node i

- Choose the split that achieves most reduction (maximizes GAIN)
- Information gain is the mutual information between the class variable and the splitting variable
- Used in the ID3 and C4.5 decision tree algorithms
- Higher Information Gain $\rightarrow$ better split

Problem with large number of partitions

- Node impurity measures tend to prefer splits that result in large number of partitions, each being small but pure



- Customer ID has highest information gain because entropy for all the children is zero

Solution: Use Gain Ratio to reduce this bias

Gain Ratio

$$\text{Gain Ratio} = \frac{\text{Information Gain}}{\text{Split Info}}$$

$$\text{Split Info} = - \sum_{i=1}^k \frac{n_i}{n} \log_2 \frac{n_i}{n}$$

Parent Node, p is split into k partitions (children)

n_i is number of records in child node i

- Adjusts Information Gain by the entropy of the partitioning.
 - ◆ Higher entropy partitioning (large number of small partitions) is penalized!
- Used in C4.5 algorithm
- Designed to overcome the disadvantage of Information Gain

Gain Ratio

$$\text{Gain Ratio}(GR) = \frac{\text{Information Gain}(IG)}{\text{Split Info}}$$

$$\text{Split Info} = - \sum_{i=1}^k \frac{n_i}{n} \log_2 \frac{n_i}{n}$$

Parent Node, p is split into k partitions (children)

n_i is number of records in child node i

	CarType		
	Family	Sports	Luxury
C1	1	4	2
C2	3	0	2

$$\text{Entropy}(\text{family}) \approx 0.811$$

$$\text{Entropy}(\text{sports}) \approx 0$$

$$\text{Entropy}(\text{luxury}) \approx 1$$

$$\text{Entropy}_{child} = \frac{4}{12} (0.811) + \frac{4}{12} (0) + \frac{4}{12} (1) = 0.603$$

$$\text{Split Info} = - \frac{4}{12} \log_2 \frac{4}{12} - \frac{4}{12} \log_2 \frac{4}{12} - \frac{4}{12} \log_2 \frac{4}{12} \approx 1.585$$

$$IG = 0.98 - 0.603 = \mathbf{0.377}$$

$$GR = \frac{0.377}{1.585} \approx \mathbf{0.238}$$

$$\text{Entropy}(\text{parent}) = - \frac{7}{12} \log_2 \frac{7}{12} - \frac{5}{12} \log_2 \frac{5}{12} \approx 0.98$$

	CarType	
	{Sports, Luxury}	{Family}
C1	6	1
C2	2	3

Binary Split: {Family} vs {Sports + Luxury}

$$Entropy(sports + Luxury) \approx 0.811$$

$$Entropy(family) \approx 0.811$$

$$Entropy_{child} = \frac{8}{12} (0.811) + \frac{4}{12} (0.811)$$

$$= 0.811$$

$$Split\ Info \approx 0.918$$

$$IG = 0.98 - 0.811 = 0.169$$

$$GR = \mathbf{0.184}$$

	CarType	
	{Sports}	{Family, Luxury}
C1	4	3
C2	0	5

Binary Split: {Sports} vs {Family+ Luxury}

$$Entropy(sports) \approx 0$$

$$Entropy(family + Luxury) \approx 0.954$$

$$Entropy_{child} = \frac{4}{12} (0) + \frac{8}{12} (0.954)$$

$$= 0.636$$

$$IG = 0.98 - 0.811 = \mathbf{0.344}$$

$$Split\ Info \approx 0.918$$

$$GR = \mathbf{0.375}$$

	CarType	
	{ Luxury }	{Family, Sports}
C1	2	5
C2	2	3

Binary Split: {Luxury} vs {Family+ Sports}

$$Entropy(Luxury) \approx 1$$

$$Entropy(Family+ Sports) \approx 0.954$$

$$Entropy_{child} = \frac{4}{12} (1) + \frac{8}{12} (0.954)$$

$$= 0.969$$

$$IG = 0.98 - 0.969 = \mathbf{0.011}$$

$$Split\ Info \approx 0.918$$

$$GR = \mathbf{0.012}$$

Best split = {Sports} vs {Family + Luxury}, because it has highest Gain Ratio

Measure of Impurity: Classification Error

- Classification error at a node t

$$Error(t) = 1 - \max_i [p_i(t)]$$

- Maximum of $1 - 1/c$ when records are equally distributed among all classes, implying the least interesting situation
- Minimum of 0 when all records belong to one class, implying the most interesting situation

Computing Error of a Single Node

$$Error(t) = 1 - \max_i [p_i(t)]$$

C1	0
C2	6

$$P(C1) = 0/6 = 0 \quad P(C2) = 6/6 = 1$$

$$Error = 1 - \max(0, 1) = 1 - 1 = 0$$

C1	1
C2	5

$$P(C1) = 1/6 \quad P(C2) = 5/6$$

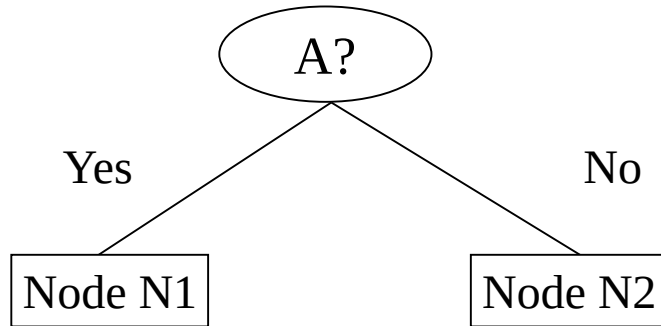
$$Error = 1 - \max(1/6, 5/6) = 1 - 5/6 = 1/6$$

C1	2
C2	4

$$P(C1) = 2/6 \quad P(C2) = 4/6$$

$$Error = 1 - \max(2/6, 4/6) = 1 - 4/6 = 1/3$$

Misclassification Error vs Gini Index



	Parent
C1	7
C2	3
Gini = 0.42	

$$\begin{aligned}
 \text{Gini}(N1) &= 1 - (3/3)^2 - (0/3)^2 \\
 &= 0
 \end{aligned}$$

$$\begin{aligned}
 \text{Gini}(N2) &= 1 - (4/7)^2 - (3/7)^2 \\
 &= 0.489
 \end{aligned}$$

	N1	N2
C1	3	4
C2	0	3
Gini=0.342		

$$\begin{aligned}
 \text{Gini(Children)} &= 3/10 * 0 \\
 &+ 7/10 * 0.489 \\
 &= 0.342
 \end{aligned}$$

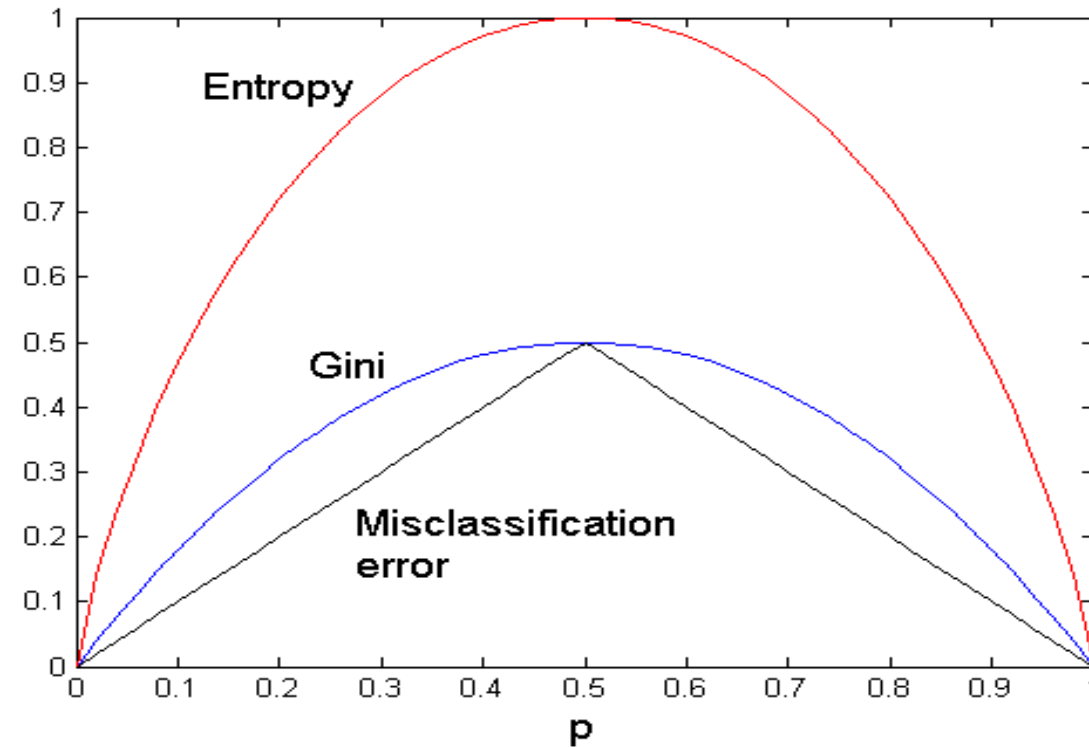
Gini improves but error remains the same!!

	N1	N2
C1	3	4
C2	1	2
Gini=0.416		

Misclassification error for all three cases = 0.3 !

Comparison among Impurity Measures

For a 2-class problem



Decision Tree Based Classification

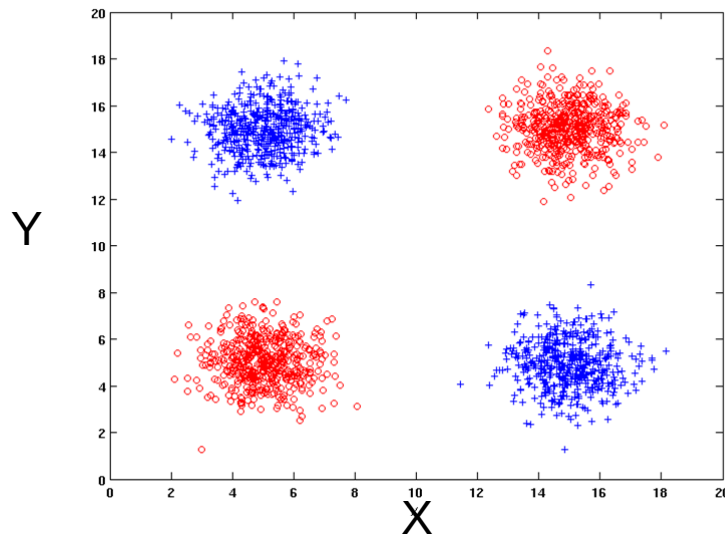
| Advantages

- Relatively inexpensive to construct
- Extremely fast at classifying unknown records
- Easy to interpret for small-sized trees
- Robust to noise (especially when methods to avoid overfitting are employed)
- Can easily handle redundant attributes
- Can easily handle irrelevant attributes (unless the attributes are interacting)

| Disadvantages

- Due to the greedy nature of splitting criterion, interacting attributes (that can distinguish between classes together but not individually) may be passed over in favor of other attributes that are less discriminating.
- Each decision boundary involves only a single attribute

Handling interactions



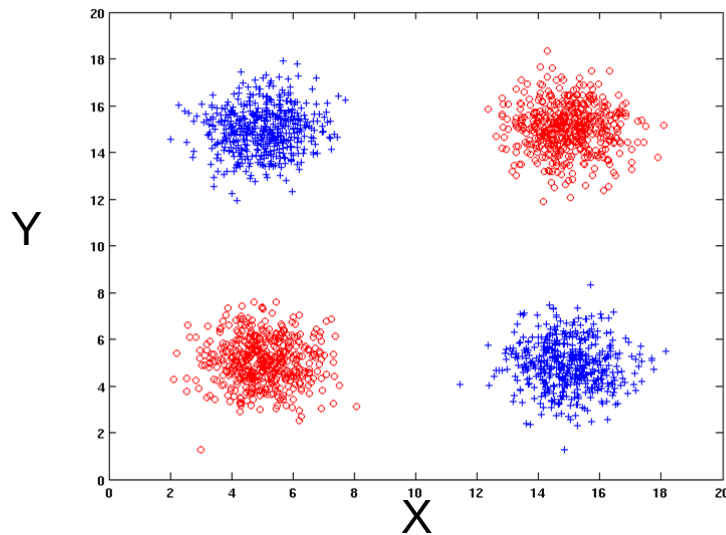
+ : 1000 instances

o : 1000 instances

Entropy (X) : 0.99

Entropy (Y) : 0.99

Handling interactions given irrelevant attributes



+ : 1000 instances

o : 1000 instances

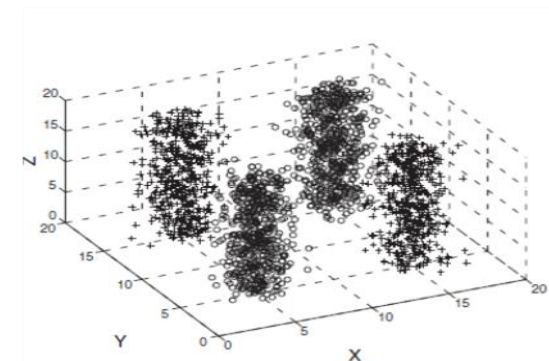
Adding Z as a noisy attribute generated from a uniform distribution

Entropy (X) : 0.99

Entropy (Y) : 0.99

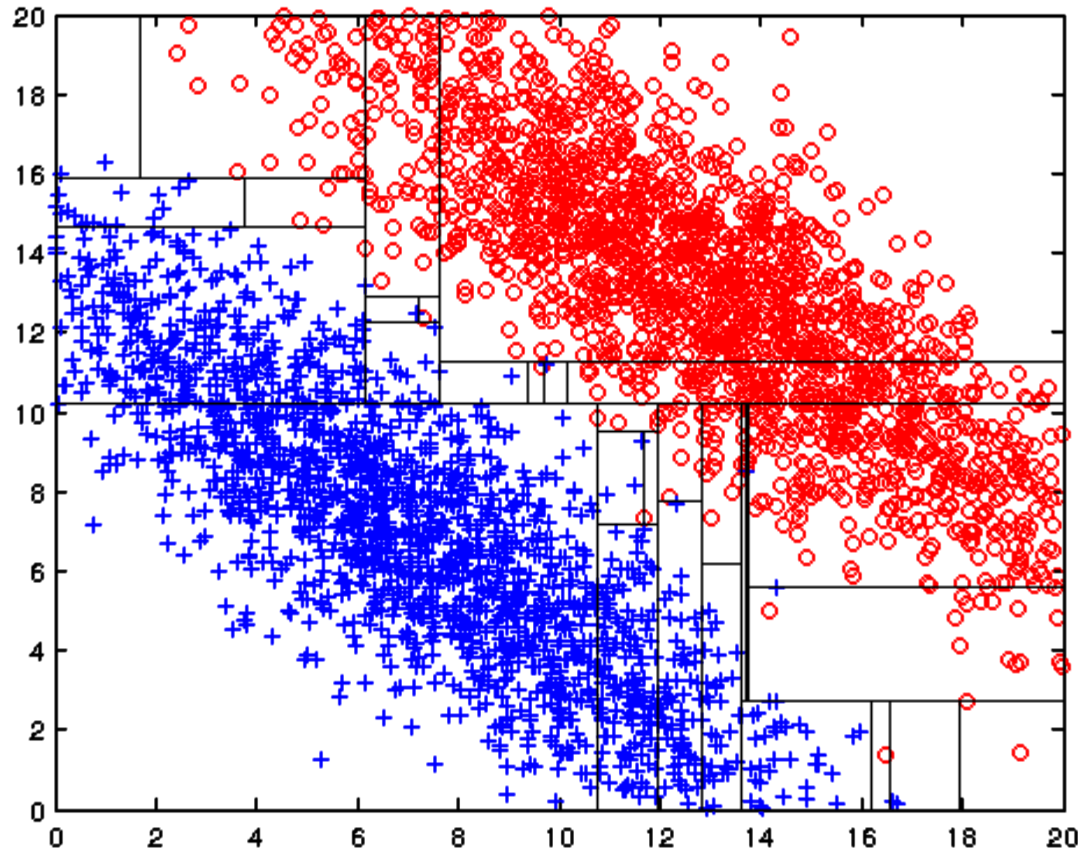
Entropy (Z) : 0.98

Attribute Z will be chosen for splitting!



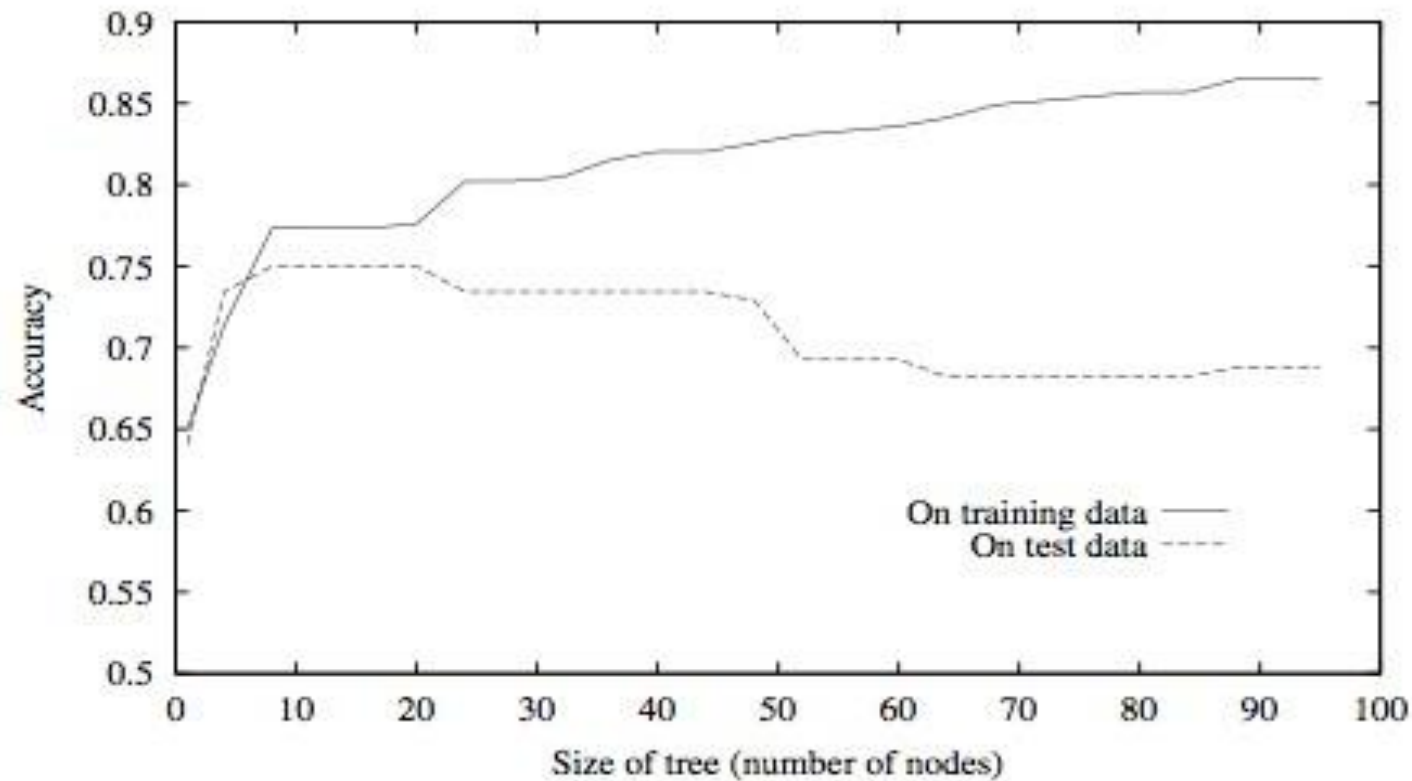
(a) Three-dimensional data with attributes X , Y , and Z .

Limitations of single attribute-based decision boundaries



Both **positive (+)** and **negative (o)** classes generated from skewed Gaussians with centers at (8,8) and (12,12) respectively.

Overfitting in decision tree learning



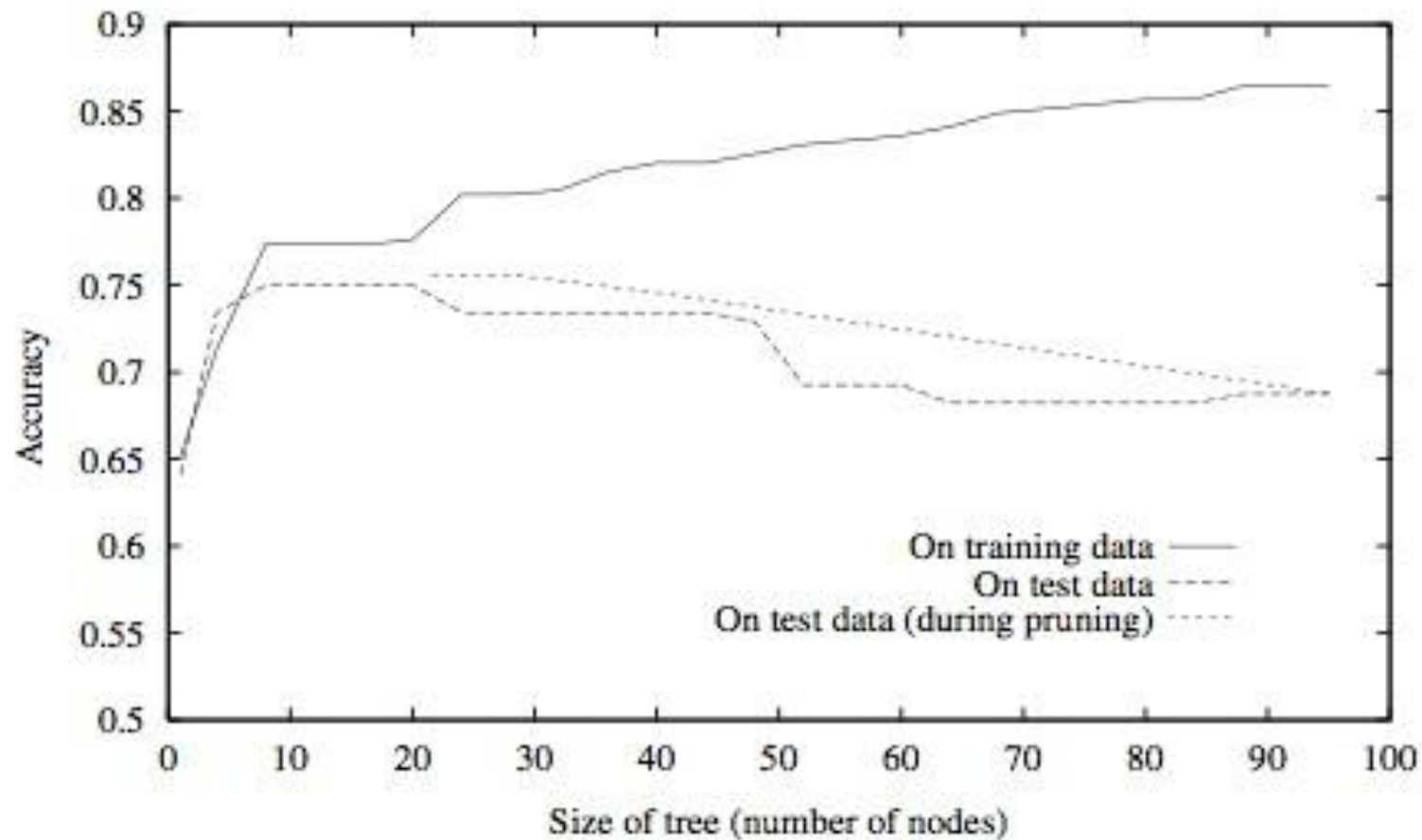
Avoid overfitting

- Two strategies
 1. Stop growing the tree earlier, before perfect classification (pre-pruning)
 2. Allow the tree to overfit the data, and then post-prune the tree (Post-pruning)
- Training and validation set
 - Split the data into **training** and **validation** sets
 - Use validation data to assess the usefulness of post-pruning
- Pruning methods
 - Reduced error pruning
 - Rule pruning
- Other approaches
 - Use a **statistical test** to estimate effect of expanding or pruning
 - **Minimum description length principle**: uses a measure of complexity of encoding the DT and the examples, and halt growing the tree when this encoding size is minimal

Reduced-error pruning (Quinlan 1987)

- Each node is a candidate for pruning
- Pruning process:
 - Remove the subtree rooted at a node
 - Replace it with a **leaf node**
 - Assign the leaf the most **common class**
- Validation-based decision:
 - Prune a node only if the resulting tree performs no worse on the validation set
- Iterative pruning:
 - At each step, prune the node whose removal most improves validation accuracy
- Stopping criterion:
 - Stop when no further pruning improves accuracy

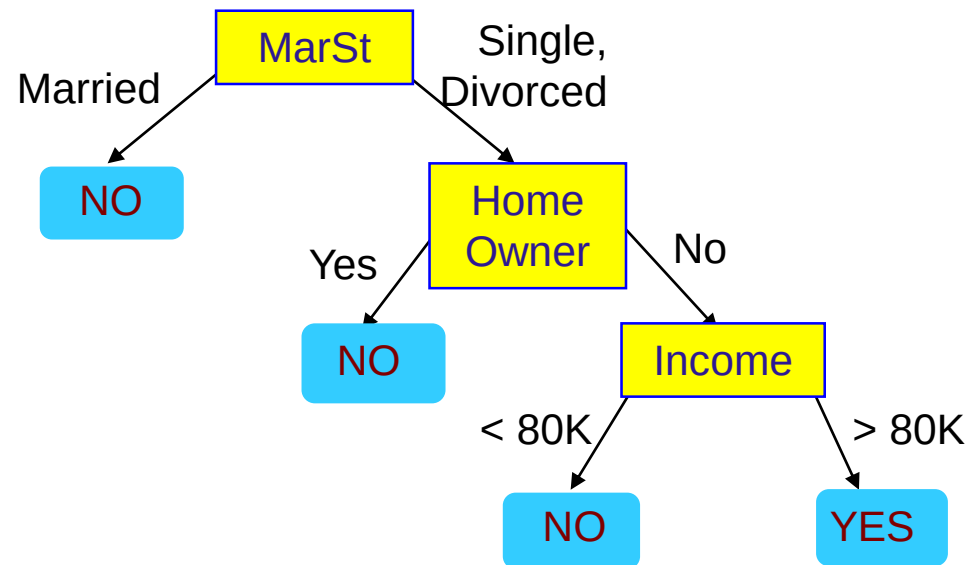
Effect of reduced error pruning



Rule post-pruning

1. Build the decision tree
 - Create the tree using the training set
2. Convert the tree into an equivalent set of rules
 - Each path $\rightarrow$ one rule
 - Each node along a path $\rightarrow$ a **pre-condition**
 - Each leaf $\rightarrow$ the class (**post-condition**)
3. Prune (generalize) each rule
 - Remove pre-conditions that do not improve accuracy
 - Evaluate using:
 - Validation set, or
 - Pessimistic/statistical inspired measure
4. Order and apply rules
 - Sort rules by estimated order of accuracy
 - Apply them sequentially to classify new instances

Converting to rules



Why converting to rules?

- Each distinct path produces a different rule: a condition removal may be based on a local (contextual) criterion. Node pruning is global and affects all the rules
- In rule form, tests are not ordered and there is no bookkeeping involved when conditions (nodes) are removed
- Converting to rules improves readability for humans

$(MarSt = Married) \Rightarrow No$

$(MarSt = Single, Divorced) \wedge (HomeOwner = Yes) \Rightarrow No$

$(MarSt = Single, Divorced) \wedge (HomeOwner = No) \wedge (Income < 80K) \Rightarrow No$

$(MarSt = Single, Divorced) \wedge (HomeOwner = No) \wedge (Income > 80K) \Rightarrow Yes$

THANK YOU